

## SKILL WEEK-3

### Session-5

**Task1:** To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
GNU nano 8.7.1
}

int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Single Quote Parsing Demonstration\n");
    printf("Enter a command or text:\n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("\nEmpty input detected.\n");
        return 0;
    }

    int count = tokenize(input, tokens);

    if (count < 0) {
        printf("Parsing failed.\n");
        return 1;
    }

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: \"%s\"", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf(" (single-quoted, literal content)");

        printf("\n");
    }

    printf("\nValidation: Parsing successful.\n");
    printf("Total tokens: %d\n", count);

    return 0;
}
```

```

#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 20

typedef struct {
    char value[100];
    int quoted;
} Token;

int tokenize(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {
        /* Skip whitespace */
        while (input[i] == ' ' || input[i] == '\t') {
            i++;
        }

        if (input[i] == '\0')
            break;

        /* Handle single-quoted string */
        if (input[i] == '\'') {
            int j = 0;
            int start = ++i;

            while (input[i] != '\0' && input[i] != '\') {
                if (j < 99)
                    tokens[count].value[j++] = input[i];
                i++;
            }

            tokens[count].value[j] = '\0';
            tokens[count].quoted = 1;

            if (input[i] == '\') {
                i++;
            } else {
                printf("\nError: Unmatched single quote.\n");
                return -1;
            }
        }
    }
}

```

```

shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ ~/OSSP
-bash: /home/shamikahasini/OSSP: Is a directory
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ cd ~/OSSP
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP$ mkdir -p skill/week3/task1
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP$ cd skill/week3/task1
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ pwd
/home/shamikahasini/OSSP/skill/week3/task1
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ ls
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ nano single_quotes.c
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ gcc single_quotes.c -o single_quotes
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$ ./single_quotes
Single Quote Parsing Demonstration
Enter a command or text:
> echo 'Hello $USER'

--- Parsing Result ---
Token 1: "echo"
Token 2: "Hello $USER" (single-quoted, literal content)

Validation: Parsing successful.
Total tokens: 2
shamikahasini@LAPTOP-D2M8NKNKBH:~/OSSP/skill/week3/task1$

```

**Observation:** The program successfully processed single-quoted strings and preserved their contents as literal text. Variable expressions such as \$USER inside single quotes were not expanded. The quoted strings were correctly stored as tokens, and the parsing results were validated successfully. The program also detected unmatched single quotes and handled empty input appropriately. Thus, single-quote handling, literal content preservation, token storage, parsing validation, and edge-case testing were successfully demonstrated.

**Task2:** To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 300
#define MAX_TOKENS 20
#define MAX_TOKEN_LENGTH 150

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    int quoted;
} Token;

void expand_variables(char *input, char *output) {
    int i = 0, j = 0;

    while (input[i] != '\0' && j < MAX_TOKEN_LENGTH - 1) {
        if (input[i] == '$') {
            i++;

            char variable[50];
            int k = 0;

            while ((input[i] >= 'A' && input[i] <= 'Z') ||
                   (input[i] >= 'a' && input[i] <= 'z') ||
                   (input[i] >= '0' && input[i] <= '9') ||
                   input[i] == '_') {
                if (k < 49)
                    variable[k++] = input[i];
                i++;
            }

            variable[k] = '\0';
            char *value = getenv(variable);

            if (value != NULL) {
                for (int x = 0; value[x] != '\0' &&
                     j < MAX_TOKEN_LENGTH - 1; x++) {
                    output[j++] = value[x];
                }
            }
            else {
                output[j++] = input[i++];
            }
        }
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int tokenize(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        while (input[i] != '\0' && input[i] != '"') {
            if (j < MAX_TOKEN_LENGTH - 1)
                temp[j++] = input[i];

            i++;
        }

        if (input[i] != '"') {
            printf("\nSyntax Error: Unmatched double quote.\n");
            return -1;
        }

        temp[j] = '\0';
        i++;

        expand_variables(temp, tokens[count].value);
        tokens[count].quoted = 1;
        count++;
    }

    /* Normal token */
    else {
        int j = 0;

        while (input[i] != '\0' &&
               input[i] != ' ' &&
               input[i] != '\t') {

            if (input[i] == '"') {
                printf("\nSyntax Error: Unexpected double quote.\n");
                return -1;
            }

            if (j < MAX_TOKEN_LENGTH - 1)
                tokens[count].value[j++] = input[i];

            i++;
        }

        tokens[count].value[j] = '\0';
        tokens[count].quoted = 0;
        count++;
    }
}

return count;
}
```

```

}

int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Double Quote Parsing Demonstration\n");
    printf("Enter a command or text:\n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("\nEmpty input detected.\n");
        return 0;
    }

    int count = tokenize(input, tokens);

    if (count < 0) {
        printf("Parsing failed.\n");
        return 1;
    }

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: \"%s\"", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf(" (double-quoted)");

        printf("\n");
    }

    printf("\nValidation: Parsing successful.\n");
    printf("Total tokens: %d\n", count);

    return 0;
}

```

```

shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ nano double_quotes.c
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ gcc double_quotes.c -o double_quotes
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ ls
double_quotes  double_quotes.c
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ ./double_quotes
Double Quote Parsing Demonstration
Enter a command or text:
> echo "Hello World $USER"

--- Parsing Result ---
Token 1: "echo"
Token 2: "Hello World madhulika" (double-quoted)

Validation: Parsing successful.
Total tokens: 2
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ ./double_quotes
Double Quote Parsing Demonstration
Enter a command or text:
> echo "Hello World
Syntax Error: Unmatched double quote.
Parsing failed.
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/skill/week3/task2$ |

```

**Observation:** The program successfully processed double-quoted strings while preserving spaces within the quoted content. Variable expansion was correctly performed, with \$USER replaced by the current username. The quoted command was parsed as a single token and the parsing result was validated successfully. The program also detected an unmatched double quote and reported a syntax error. Thus, double-quote handling, space preservation, variable expansion, quoted command parsing, output validation, and error detection were successfully demonstrated.

## Session-6

**Task1:** To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300
#define MAX_TOKENS 30
#define MAX_TOKEN_LENGTH 100

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    int escaped;
} Token;

int tokenize(char *input, Token tokens[]) {
    int i = 0;
    int count = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {

        /* Skip normal whitespace */
        while (input[i] == ' ' || input[i] == '\t')
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;
        int escaped = 0;

        while (input[i] != '\0' &&
               input[i] != ' ' &&
               input[i] != '\t') {

            /* Handle escape character */
            if (input[i] == '\\') {
                i++;

                if (input[i] != '\0') {
                    if (j < MAX_TOKEN_LENGTH - 1)
                        tokens[count].value[j++] = input[i];

                    escaped = 1;
                    i++;
                }
            }

            else {
                if (j < MAX_TOKEN_LENGTH - 1)
                    tokens[count].value[j++] = input[i];

                i++;
            }
        }
    }
}
```



```

    }
}

tokens[count].value[j] = '\0';
tokens[count].escaped = escaped;
count++;
}

return count;
}

int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Escape Sequence Parsing Demonstration\n");
    printf("Enter a command or complex input:\n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("\nEmpty input detected.\n");
        return 0;
    }

    int count = tokenize(input, tokens);

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: \"%s\"", i + 1, tokens[i].value);

        if (tokens[i].escaped)
            printf(" (escaped characters handled)");

        printf("\n");
    }

    printf("\nValidation: Parser output is valid.\n");
    printf("Total tokens: %d\n", count);
    printf("Escape sequence processing completed successfully.\n");

    return 0;
}

```

```

shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ cd ~/OSSP
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP$ mkdir -p skill/week3/task1
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP$ cd skill/week3/task1
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ pwd
/home/shamikahasini/OSSP/skill/week3/task1
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ ls
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ nano single_quotes.c
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ gcc single_quotes.c -o single_quotes
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ ls
escape_parser  escape_parser.c  single_quotes  single_quotes.c
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ ./escape_parser
Escape Sequence Parsing Demonstration
Enter a command or complex input:
> echo Hello World

--- Parsing Result ---
Token 1: "echo"
Token 2: "Hello World" (escaped characters handled)

Validation: Parser output is valid.
Total tokens: 2
Escape sequence processing completed successfully.
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ ./escape_parser
Escape Sequence Parsing Demonstration
Enter a command or complex input:
> echo Hello\ World\ &\ Test

--- Parsing Result ---
Token 1: "echo"
Token 2: "Hello World & Test" (escaped characters handled)

Validation: Parser output is valid.
Total tokens: 2
Escape sequence processing completed successfully.
shamikahasini@LAPTOP-D2M8NKK8H:~/OSSP/skill/week3/task1$ |

```

**Observation:** The program successfully processed escape sequences, handled escaped spaces and special symbols, and preserved the intended characters in the input. The parser correctly generated tokens from simple and complex inputs, and the resulting parser output was validated successfully. Thus, escape-sequence handling and parsing of complex inputs were successfully demonstrated.

**Task2:** To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int argc = 0;

    printf("Process Creation and Command Launching Demonstration\n");
    printf("Enter a command with arguments:\n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command entered.\n");
        return 0;
    }

    /* Split command into arguments */
    char *token = strtok(input, " \t");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }
}
```

```

        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("Child Process PID: %d\n", getpid());
        printf("Executing command: %s\n", args[0]);

        execvp(args[0], args);

        /* Only reached if execvp fails */
        perror("Execution failed");
        exit(EXIT_FAILURE);
    }

    else {
        /* Parent process */
        int status;

        printf("Parent waiting for child process...\n");

        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid failed");
            return 1;
        }

        if (WIFEXITED(status)) {
            printf("Child process exited with status: %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
|

```



```
shamikahasini@LAPTOP-D2M8NKKKBH:~$ cd ~/OSSP
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP$ mkdir -p skill/week3/task2
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP$ cd skill/week3/task2
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ pwd
/home/shamikahasini/OSSP/skill/week3/task2
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ ls
double_quotes.c
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ nano process_launcher.c
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ gcc process_launcher.c -o process_launcher
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ ls
double_quotes  double_quotes.c  process_launcher  process_launcher.c
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$ ./process_launcher
Process Creation and Command Launching Demonstration
Enter a command with arguments:
> ls -l

Parent Process PID: 1349
Parent waiting for child process...
Child Process PID: 1350
Executing command: ls
total 44
-rwxr-xr-x 1 shamikahasini shamikahasini 16344 Aug 11 13:55 double_quotes
-rw-r--r-- 1 shamikahasini shamikahasini 3609 Aug 11 13:54 double_quotes.c
-rwxr-xr-x 1 shamikahasini shamikahasini 16480 Aug 11 14:12 process_launcher
-rw-r--r-- 1 shamikahasini shamikahasini 1755 Aug 11 14:12 process_launcher.c
Child process exited with status: 0
Parent process completed.
shamikahasini@LAPTOP-D2M8NKKKBH:~/OSSP/skill/week3/task2$
```

**Observation:** The program successfully created a child process using `fork()` and executed commands using `execvp()`. Command-line arguments were passed correctly to the child process, while the parent process waited for the child to complete using `waitpid()`. Invalid commands were detected and appropriate execution errors were displayed. Thus, child process creation, program execution, argument passing, parent process management, error handling, and command launching were successfully demonstrated.